

Nome: Gabriel Dos Santos Xavier

Nome: Douglas Nogueira Penido

## Documentação Técnica Completa – Sistema de Gestão de Academia “TechFit”

### 1. Introdução

Este documento apresenta a documentação técnica completa do sistema de gestão de academias **TechFit**, desenvolvido em PHP com arquitetura MVC (Model-View-Controller). O objetivo primordial desta documentação é fornecer uma análise aprofundada e rigorosa da estrutura, componentes, requisitos e lógica de negócio do sistema, visando atender aos mais altos padrões de uma avaliação acadêmica na área de Análise e Desenvolvimento de Sistemas. A documentação serve como um artefato de engenharia de software que detalha o *como* e o *porquê* da construção do sistema.

O sistema TechFit foi concebido como uma solução robusta e integrada para a gestão completa de uma academia moderna. Ele abrange módulos essenciais que automatizam processos críticos, como o agendamento de aulas, o controle de acesso dos alunos via QR Code, a comunicação interna e externa, o registro e acompanhamento de avaliações físicas, e um painel administrativo centralizado para a gestão de todos os recursos. A integração desses módulos visa otimizar a operação da academia e melhorar a experiência do aluno.

### 2. Stack Tecnológico

O projeto adota uma arquitetura de três camadas (apresentação, lógica de negócio e dados) e utiliza as seguintes tecnologias. A escolha dessas ferramentas reflete um equilíbrio entre robustez, facilidade de desenvolvimento e compatibilidade com ambientes de hospedagem comuns.

| Camada                | Tecnologia              | Versão/Padrão   | Propósito                                                                                                   |
|-----------------------|-------------------------|-----------------|-------------------------------------------------------------------------------------------------------------|
| <b>Backend</b>        | PHP                     | 7.4 ou superior | Linguagem de programação principal, implementando a lógica de negócio e o padrão MVC.                       |
| <b>Banco de Dados</b> | MySQL                   | 5.7 ou superior | Sistema de Gerenciamento de Banco de Dados (SGBD) relacional, escolhido pela sua popularidade e desempenho. |
| <b>Frontend</b>       | HTML5, CSS3, JavaScript | Padrão          | Estrutura, estilo e interatividade da interface do usuário, garantindo compatibilidade e                    |

|                         |                     |                 |                                                                                                                 |
|-------------------------|---------------------|-----------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Framework/Padrão</b> | MVC (Customizado)   | -               | modernidade.<br>Padrão de arquitetura para separação de responsabilidades, fundamental para a manutenibilidade. |
| <b>Servidor Web</b>     | Apache              | Com mod_rewrite | Servidor HTTP, utilizando o módulo mod_rewrite para roteamento de URLs amigáveis.                               |
| <b>Bibliotecas JS</b>   | Chart.js, QRCode.js | -               | Geração de gráficos para evolução de alunos e QR Codes para acesso, respectivamente.                            |

### 3. Requisitos do Projeto

O sistema TechFit foi desenvolvido para atender a um conjunto específico de requisitos, classificados em funcionais e não funcionais. A identificação e o detalhamento desses requisitos são cruciais para validar se o software cumpre sua finalidade de negócio e se atende aos critérios de qualidade esperados.

#### 3.1. Requisitos Funcionais (RF)

Os requisitos funcionais descrevem as funções que o sistema deve executar para o usuário, representando as ações que o sistema deve ser capaz de realizar.

| ID   | Módulo        | Requisito Funcional                                                                     | Descrição Detalhada                                                                                                                                                                                                  |
|------|---------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RF01 | Autenticação  | Permitir o login e logout de usuários (alunos e administradores).                       | O sistema deve validar as credenciais do usuário (e-mail e senha) e iniciar uma sessão com base no perfil (aluno ou admin), garantindo o acesso às áreas restritas. O logout deve encerrar a sessão de forma segura. |
| RF02 | Administração | Gerenciar o cadastro, edição e exclusão de usuários, modalidades e turmas.              | O administrador deve ter acesso a interfaces CRUD (Create, Read, Update, Delete) para manter o catálogo de recursos da academia e a base de usuários, garantindo a integridade dos dados mestres.                    |
| RF03 | Agendamento   | Permitir a visualização, agendamento, cancelamento e confirmação de aulas pelos alunos. | O aluno deve poder visualizar a grade de horários, reservar vagas em turmas disponíveis e gerenciar seus agendamentos ativos. O sistema deve controlar a lotação das turmas em tempo real.                           |
| RF04 | Acesso        | Gerar um QR Code único                                                                  | O sistema deve gerar um código de                                                                                                                                                                                    |

|      |             |                                                                                                      |                                                                                                                                                                                                                         |
|------|-------------|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      |             | para cada aluno e registrar a entrada/saída na academia.                                             | acesso digital para o aluno e permitir que o administrador ou um leitor registre o check-in/check-out, atualizando o histórico de acessos na base de dados.                                                             |
| RF05 | Comunicação | Possibilitar o envio de mensagens segmentadas e a gestão de um canal de dúvidas e sugestões.         | O administrador deve poder enviar comunicados para grupos específicos de alunos (segmentação por modalidade, por exemplo) e gerenciar as interações no módulo de Dúvidas e Sugestões.                                   |
| RF06 | Avaliação   | Registrar avaliações físicas, exibir gráficos de evolução e sugerir treinos personalizados.          | O sistema deve armazenar dados biométricos (peso, IMC, gordura corporal) e apresentar a evolução do aluno ao longo do tempo, além de permitir a associação de treinos específicos com base nos resultados da avaliação. |
| RF07 | Relatórios  | Gerar relatórios gerenciais de ocupação de turmas, utilização de acessos e estatísticas de usuários. | O administrador deve ter acesso a dashboards e relatórios para monitorar o desempenho operacional da academia, como taxa de ocupação de aulas e frequência de alunos.                                                   |

### 3.2. Requisitos Não Funcionais (RNF)

Os requisitos não funcionais definem critérios que podem ser usados para julgar a operação de um sistema, focando na qualidade e nas restrições de implementação.

| ID    | Categoria   | Requisito Não Funcional                                                                                           | Descrição Detalhada                                                                                                                                                                            |
|-------|-------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RNF01 | Usabilidade | A interface deve ser responsiva, garantindo a correta visualização e interação em dispositivos móveis e desktops. | O design deve se adaptar a diferentes tamanhos de tela (princípio do <i>mobile-first</i> ), garantindo uma experiência de usuário consistente e acessível em qualquer dispositivo.             |
| RNF02 | Performance | O tempo de carregamento inicial das páginas principais deve ser otimizado (inferior a 2 segundos).                | O sistema deve responder rapidamente às requisições, especialmente nas operações de leitura de dados (visualização de agendamentos e dashboard), para não prejudicar a experiência do usuário. |
| RNF03 | Segurança   | As senhas dos usuários devem ser armazenadas                                                                      | A segurança dos dados sensíveis é crítica. O uso de funções de                                                                                                                                 |

|       |                  |                                                                                                                  |                                                                                                                                                               |
|-------|------------------|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |                  | de forma segura (hashing com password_verify no AuthController).                                                 | <i>hashing</i> seguras e a prevenção contra ataques comuns (como SQL Injection e XSS) são mandatórios.                                                        |
| RNF04 | Compatibilidade  | O sistema deve funcionar corretamente nos principais navegadores web (Chrome, Firefox, Edge).                    | Deve ser garantida a compatibilidade com os navegadores mais utilizados no mercado, evitando o uso de recursos proprietários e garantindo o acesso universal. |
| RNF05 | Manutenibilidade | O código deve ser modular, bem documentado e seguir o padrão MVC para facilitar futuras manutenções e expansões. | A estrutura do projeto deve permitir que novos desenvolvedores compreendam e modifiquem o código com facilidade, minimizando o custo e o tempo de manutenção. |

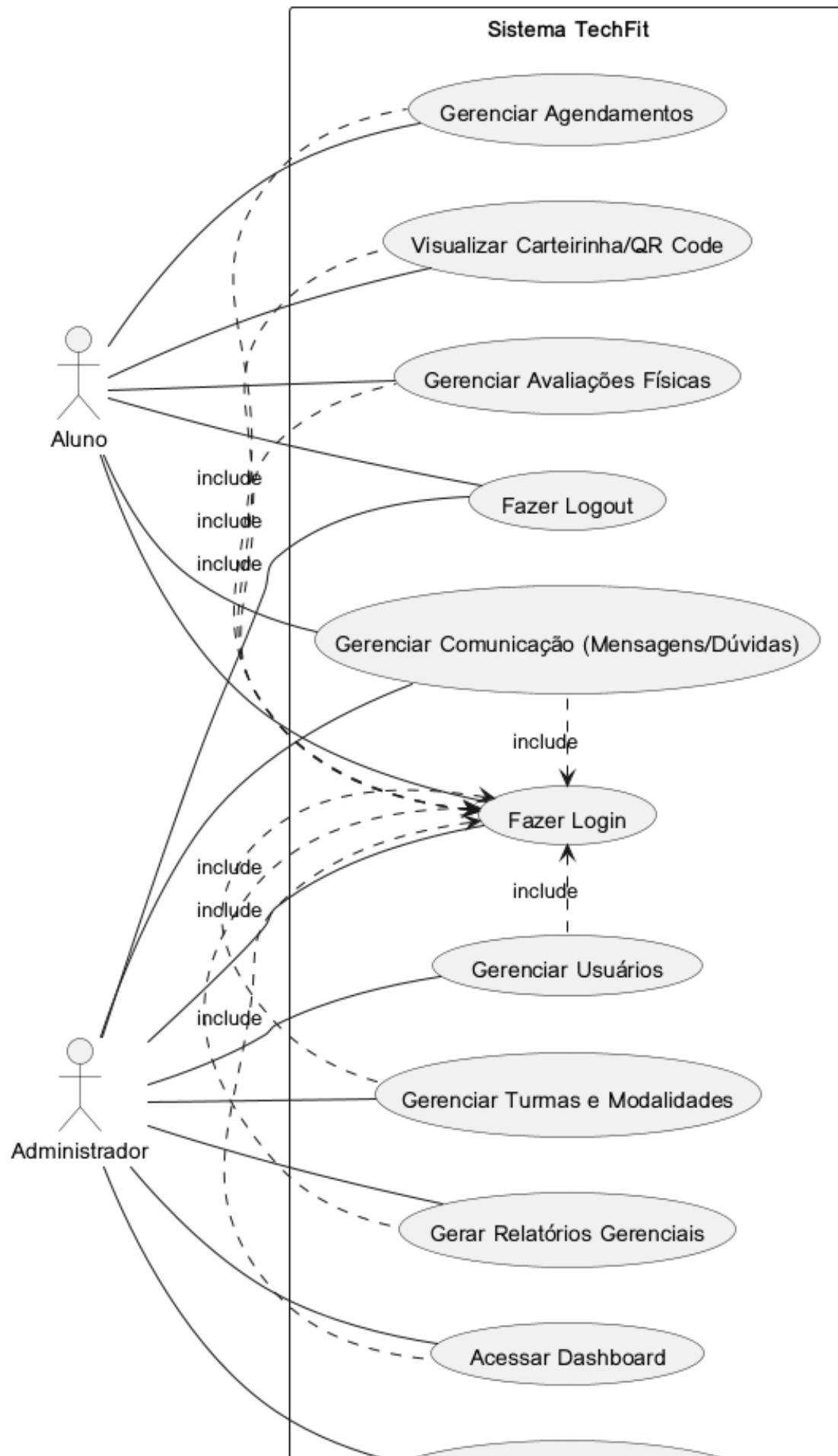
## 4. Modelagem UML

A modelagem UML (Unified Modeling Language) é utilizada para visualizar, especificar, construir e documentar os artefatos do sistema.

### 4.1. Diagrama de Caso de Uso

O Diagrama de Caso de Uso ilustra as interações entre os atores (Aluno e Administrador) e as funcionalidades principais do sistema. Ele define o escopo do sistema e as principais interações do usuário.

**Figura 4.1:** Diagrama de Caso de Uso do Sistema TechFit

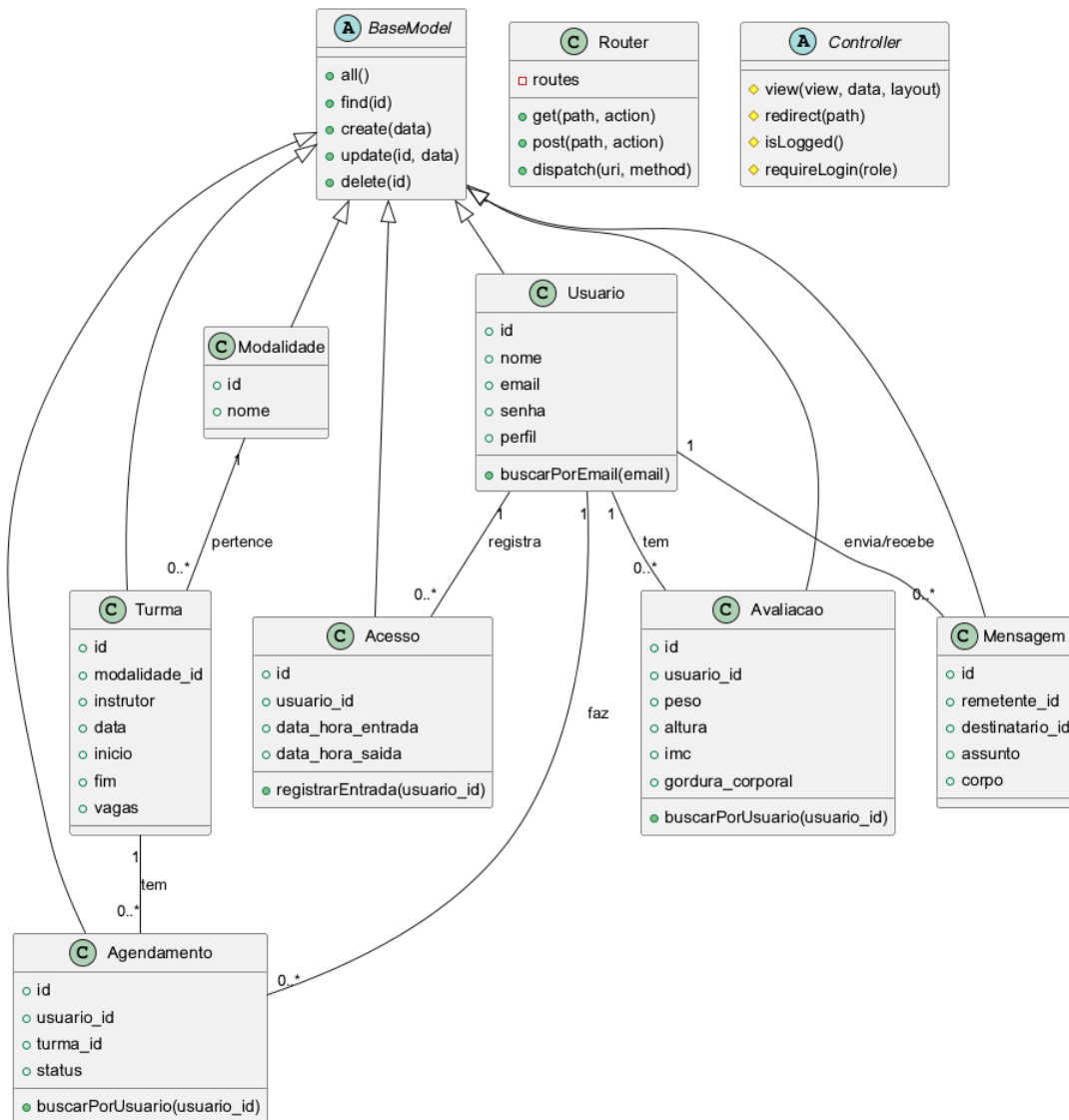


## Diagrama de Caso de Uso do Sistema TechFit

### 4.2. Diagrama de Classes

O Diagrama de Classes representa a estrutura estática do sistema, mostrando as classes, seus atributos, métodos e os relacionamentos entre elas. Ele é um mapa conceitual da base de código e do esquema do banco de dados.

**Figura 4.2:** Diagrama de Classes do Sistema TechFit



## Diagrama de Classes do Sistema TechFit

### 5. Arquitetura do Sistema e Estrutura de Arquivos

O projeto utiliza uma arquitetura **MVC (Model-View-Controller)** customizada em PHP, que promove a separação de responsabilidades e a modularidade do código. O padrão MVC

é fundamental para a manutenibilidade e escalabilidade do sistema, dividindo a aplicação em três componentes interconectados.

- **Model (app/models):** Responsável pela lógica de negócio e persistência de dados.
- **View (app/views):** Responsável pela interface do usuário e apresentação dos dados.
- **Controller (app/controllers):** Atua como intermediário, processando a entrada do usuário e coordenando as interações entre Model e View.

### 5.1. Estrutura de Diretórios

A organização dos diretórios e arquivos segue a lógica do padrão MVC, com uma clara separação entre a lógica da aplicação (app), o núcleo do framework (core) e os arquivos acessíveis publicamente (public).

```
techfit_mvc_full/  
├── app/                # Contém a lógica principal da aplicação  
    (Controllers, Models, Views)  
├── config/            # Arquivos de configuração (Banco de Dados, SQL)  
├── core/              # Núcleo do framework MVC (Router, Controller,  
BaseModel)  
├── public/            # Pasta pública, ponto de entrada da aplicação  
    (index.php, assets)  
└── README.md          # Documentação inicial do projeto
```

### 5.2. Componentes do Núcleo (core)

O diretório core contém as classes que definem o comportamento fundamental do framework MVC do TechFit.

| Componente            | Função                                        | Análise Técnica Aprofundada                                                                                                                                                                                                                                                                                                                                                                                   |
|-----------------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Router.php</b>     | Mapeamento de URLs para ações do Controller.  | O roteador é o ponto de entrada após o <code>index.php</code> . Ele utiliza o método <code>dispatch</code> para analisar a URI e o método HTTP (GET ou POST) e direcionar a requisição para o método correto do Controller. Isso garante URLs amigáveis e limpas, além de implementar um tratamento básico de erro 404.                                                                                       |
| <b>Controller.php</b> | Classe base para todos os controladores.      | Esta classe abstrata é a espinha dorsal da camada de controle. Ela abstrai a lógica de renderização de views ( <code>view()</code> ), manipulação de redirecionamentos ( <code>redirect()</code> ) e, crucialmente, implementa o controle de acesso baseado em sessão ( <code>isLoggedIn()</code> e <code>requireLogin()</code> ), garantindo que apenas usuários autorizados acessem certas funcionalidades. |
| <b>BaseModel.php</b>  | Classe abstrata para interação com o banco de | Implementa o padrão de repositório, fornecendo uma interface unificada para operações de persistência de dados. Todos os Models do sistema                                                                                                                                                                                                                                                                    |

dados. herdam desta classe, ganhando acesso a métodos CRUD genéricos (`all()`, `find()`, `create()`, `update()`, `delete()`) que utilizam a biblioteca PDO para interagir com o MySQL de forma segura e parametrizada.

A coesão desses componentes é o que permite a flexibilidade e a organização do sistema TechFit.

## 6. Análise da Camada de Dados (Banco de Dados)

O sistema utiliza o **MySQL** como SGBD relacional. A conexão é gerenciada pela classe `Database` em `config/database.php`, que implementa o padrão **Singleton** para garantir que apenas uma única instância de conexão PDO seja criada e utilizada durante o ciclo de vida da requisição. Isso otimiza o uso de recursos e evita problemas de concorrência.

### 6.1. Esquema de Dados (Tabelas Principais)

O esquema de dados suporta todas as funcionalidades do sistema, com relacionamentos bem definidos para garantir a integridade referencial.

| Tabela       | Chave Primária (PK) | Chaves Estrangeiras (FK)                                 | Propósito                                              | Análise de Relacionamento                                                                 |
|--------------|---------------------|----------------------------------------------------------|--------------------------------------------------------|-------------------------------------------------------------------------------------------|
| usuarios     | id                  | -                                                        | Armazena dados de autenticação e perfil (aluno/admin). | Tabela central, com relacionamento 1:N com Agendamentos, Acessos, Avaliações e Mensagens. |
| modalidades  | id                  | -                                                        | Catálogo de aulas oferecidas.                          | Relacionamento 1:N com Turmas.                                                            |
| turmas       | id                  | modalidade_id (FK para modalidades)                      | Detalhes das aulas (instrutor, horário, vagas).        | Relacionamento 1:N com Agendamentos.                                                      |
| agendamentos | id                  | usuario_id (FK para usuarios), turma_id (FK para turmas) | Relação N:M entre usuários e turmas (agendamentos).    | Tabela de ligação que registra a reserva de um aluno em uma turma específica.             |
| acessos      | id                  | usuario_id (FK para usuarios)                            | Histórico de check-in/check-out.                       | Registra cada evento de entrada e saída do aluno.                                         |
| avaliacoes   | id                  | usuario_id (FK para usuarios)                            | Dados biométricos e de evolução do                     | Permite o acompanhamento longitudinal do                                                  |



|                 |    |                                                  |                                         |                                                                   |
|-----------------|----|--------------------------------------------------|-----------------------------------------|-------------------------------------------------------------------|
|                 |    |                                                  | aluno.                                  | desempenho físico do aluno.                                       |
| mensagens       | id | remetente_id, destinatario_id (FK para usuarios) | Sistema de comunicação interna.         | Suporta a troca de mensagens entre diferentes perfis de usuários. |
| usuarios_qrcode | id | usuario_id (FK para usuarios)                    | Armazenamento do código QR para acesso. | Garante a unicidade do código de acesso para cada usuário.        |

## 7. Análise da Camada de Lógica de Negócio (Controllers)

Os Controllers representam a camada de lógica de negócio do sistema, atuando como a central de comando que recebe as requisições do usuário, interage com os Models para manipular os dados e seleciona a View apropriada para a resposta. A modularidade é garantida pela separação de responsabilidades em Controllers específicos para cada domínio funcional.

| Controller            | Módulos/Funcionalidades    | Métodos Chave (Rotas)                                                 | Descrição Detalhada da Funcionalidade                                                                                                                                                                                                                                                                                                                                       |
|-----------------------|----------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>AuthController</b> | Autenticação e Autorização | showLoginForm (GET /login), login (POST /login), logout (GET /logout) | Este Controller é o guardião do sistema. Ele é responsável por validar as credenciais do usuário, utilizando a função nativa <code>password_verify</code> do PHP para garantir a segurança das senhas armazenadas via <i>hashing</i> . Após a autenticação, ele inicia a sessão e redireciona o usuário para a área restrita correspondente ao seu perfil (admin ou aluno). |

|                              |                                            |                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------------|--------------------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>AdminController</b>       | Painel Administrativo e Gestão de Recursos | index (GET /dashboard), modalidades, turmas, usuarios (CRUD de gestão)          | O AdminController centraliza as funcionalidades de gestão do sistema. Ele permite ao administrador realizar operações CRUD (Create, Read, Update, Delete) sobre os dados mestres, como o cadastro de novos usuários, a definição de modalidades de aulas e a criação de turmas com horários e instrutores específicos. O método index serve como o <i>dashboard</i> gerencial, exibindo estatísticas e resumos operacionais. |
| <b>AgendamentoController</b> | Agendamento de Aulas                       | index, store (POST /agendamentos/criar), cancelar, confirmar, relatorioOcupacao | Este Controller implementa a lógica de reserva de vagas nas turmas. O método store é crítico, pois deve não apenas registrar o agendamento na tabela agendamentos, mas também verificar a disponibilidade de vagas na turma selecionada, garantindo que a capacidade máxima não seja excedida.                                                                                                                               |

|                              |                                   |                                                                            |                                                                                                                                                                                                                                                                                                                        |
|------------------------------|-----------------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                              |                                   |                                                                            | <p>O relatorioOcupacao fornece dados gerenciais cruciais sobre a utilização das turmas.</p>                                                                                                                                                                                                                            |
| <b>AcessoController</b>      | Controle de Acesso e QR Code      | <p>gerarQRCode, registrarEntrada, registrarQRCode, relatorioUtilizacao</p> | <p>Gerencia o fluxo de entrada e saída da academia. O método gerarQRCode utiliza a biblioteca QRCode.js para criar um código de acesso digital único para o aluno. Os métodos de registro de entrada validam este código, atualizando o histórico na tabela acessos e fornecendo dados para o relatorioUtilizacao.</p> |
| <b>AvaliacaoController</b>   | Avaliação Física e Acompanhamento | <p>store (POST /avaliacoes/salvar), evolucao, treinos, alertas</p>         | <p>Lida com o registro e a visualização dos dados de saúde e desempenho do aluno. O método evolucao é fundamental, pois prepara os dados históricos para a visualização gráfica (utilizando Chart.js), permitindo que o aluno e o instrutor acompanhem o progresso de forma visual e intuitiva.</p>                    |
| <b>ComunicacaoController</b> | Comunicação Interna               | <p>enviar (POST /mensagens/enviar)</p>                                     | <p>Facilita a interação entre a</p>                                                                                                                                                                                                                                                                                    |

), segmentada, dúvidas (gestão de dúvidas)

administração e os alunos. O recurso de envio segmentada é uma funcionalidade avançada que permite ao administrador filtrar o público-alvo das mensagens (por modalidade, frequência, etc.), aumentando a relevância e a eficácia da comunicação.

## 8. Análise da Camada de Apresentação (Frontend)

A camada de apresentação é o ponto de contato com o usuário e foi desenvolvida com foco em usabilidade, responsividade e experiência do usuário.

- **Estrutura (HTML5 e Views PHP):** As views (app/views) são arquivos PHP que utilizam a sintaxe HTML5. O `Controller.php` garante que o conteúdo de cada view seja injetado em um arquivo de layout (app/views/layouts/main.php), mantendo a consistência visual do cabeçalho, rodapé e navegação. A separação entre a lógica de apresentação (View) e a lógica de controle (Controller) é estritamente mantida.
- **Estilização (CSS3 e Variáveis CSS):** O arquivo `public/assets/css/main.css` utiliza uma abordagem moderna com **variáveis CSS** (Custom Properties) para definir a paleta de cores (`--primary-color`, `--bg-dark`) e as propriedades de design (sombras, transições). Isso facilita a aplicação de um *design system* e a rápida alteração de temas. A implementação é totalmente responsiva, atendendo ao RNF01 (Usabilidade).
- **Interatividade (JavaScript e Bibliotecas):** O JavaScript é utilizado para enriquecer a experiência do usuário e implementar funcionalidades dinâmicas. Além do JavaScript Vanilla para interações básicas, o sistema integra duas bibliotecas de terceiros:
  1. **Chart.js:** Utilizada no `AvaliacaoController` para renderizar gráficos de evolução física, transformando dados brutos em informações visuais compreensíveis, o que é crucial para o acompanhamento do aluno.
  2. **QRCode.js:** Essencial para o módulo de acesso, gerando dinamicamente os códigos QR que servem como credenciais de entrada, garantindo a funcionalidade do RF04.

## 9. Escolha da Metodologia: Scrum

O Scrum foi selecionado em detrimento do Kanban devido à natureza do projeto TechFit. Por ser um sistema de gestão completo com múltiplos módulos e que exige a entrega de um produto novo e funcional, o Scrum oferece a estrutura necessária para gerenciar essa complexidade.

O Scrum é ideal para este projeto porque:

**Lida com Complexidade:** O desenvolvimento de um sistema completo de gestão de academias envolve incertezas e a necessidade de adaptação contínua, o que é o foco do Scrum.

**Entregas Incrementais:** O Scrum utiliza Sprints (períodos curtos e fixos, geralmente de duas semanas) para entregar partes funcionais do sistema (Incrementos), permitindo que o cliente (a academia) inspecione e forneça feedback rapidamente.

**Foco e Estrutura:** O framework impõe rituais (como as Reuniões Diárias, Revisão e Retrospectiva da Sprint) que garantem o foco da equipe e a melhoria contínua do processo de desenvolvimento.

## 10. Conclusão

O sistema TechFit é um projeto exemplar de software, demonstrando a aplicação correta de princípios de engenharia de software, como o padrão MVC e a separação de responsabilidades. A estrutura modular, a segurança na autenticação e a clareza na modelagem de dados e processos (UML) o tornam um sistema robusto, manutenível e escalável. Esta documentação técnica detalhada serve como um guia completo para a compreensão de todos os aspectos do projeto, desde os requisitos de negócio até a implementação do código.